

Software Design Specifications

for

FIFA World Cup Analytics

Prepared by:

Team 2

Roll No: SE23UARI141, SE23UARI105, SE23UARI011, SE23UARI102,
SE23UARI113

B.Tech Artificial Intelligence, Mahindra University

April 8th, 2026

Document Information

Title	FIFA World Cup Analytics – SDS
Project Manager	Team 2
Document Version No.	1.0
Document Version Date	April 8, 2026
Prepared By	Team 2
Preparation Date	April 8, 2026

Version History

Ver.	Date	Revised By	Description	Filename
1	Jan 10, 2026	Team 2	Defined project scope, analytics methodology, and data source selection	sds_v1
2	Jan 24, 2026	Team 2	Implemented CSV data pipeline and core statistical aggregations in <code>data_processor.py</code>	sds_v2
3	Feb 07, 2026	Team 2	Built Flask backend with all six REST API endpoints and JSON response contracts	sds_v3
4	Feb 21, 2026	Team 2	Integrated Chart.js visualisations; connected frontend to live API endpoints	sds_v4
5	Mar 08, 2026	Team 2	Refined dark theme UI, added glassmorphism cards, and improved responsiveness	sds_v5
6	Mar 31, 2026	Team 2	Final release; completed SDS documentation and deployed application	sds_v6

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Definitions, Acronyms, and Abbreviations	3
1.4	References	3
2	Use Case View	3
2.1	Use Case 1: View Goals Trend Over Time	3
2.2	Use Case 2: Compare Two Teams Head-to-Head	4
2.3	Use Case 3: Explore Top Teams by Metric	4
2.4	Use Case 4: View Goal Distribution by Continent	4
3	Design Overview	5
3.1	Design Goals and Constraints	5
3.2	Design Assumptions	5
3.3	Significant Design Packages	5
3.4	Dependent External Interfaces	6
3.5	Implemented Application External Interfaces	6
4	Logical View	6
4.1	Design Model	6
4.2	Use Case Realization	7
5	Data View	8
5.1	Domain Model	8
5.2	Data Model (Persistent Data View)	8
5.2.1	Data Dictionary	8
6	Exception Handling	8
7	Configurable Parameters	9
8	Quality of Service	9
8.1	Availability	9
8.2	Security and Authorization	9
8.3	Load and Performance Implications	10
8.4	Monitoring and Control	10

1 Introduction

1.1 Purpose

This Software Design Specifications (SDS) document describes the internal design of the FIFA World Cup Analytics. It is intended for the development team, software engineering course evaluators, and any future contributors to the project. The document details the system architecture, component decomposition, data model, and quality attributes of the application.

1.2 Scope

This document applies to the FIFA World Cup Analytics , an interactive web-based visual analytics platform hosted at <https://github.com/prithvi-kaizen/fifa-wc-analytics>. The platform allows users to explore historical FIFA World Cup data spanning 1930 to 2022 through interactive charts and statistical summaries. The scope covers the Flask backend, the data processing layer, the REST API, and the frontend visualization layer.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SDS	Software Design Specifications
API	Application Programming Interface
REST	Representational State Transfer
CSV	Comma-Separated Values
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
JSON	JavaScript Object Notation
WC	World Cup

1.4 References

- Project Repository: <https://github.com/prithvi-kaizen/fifa-wc-analytics>
- Flask Documentation: <https://flask.palletsprojects.com>
- Chart.js Documentation: <https://www.chartjs.org/docs>
- Pandas Documentation: <https://pandas.pydata.org/docs>

2 Use Case View

The following use cases represent the primary functional scenarios of the FIFA World Cup Analytics , each of which exercises multiple design components.

2.1 Use Case 1: View Goals Trend Over Time

Description: The user loads the and views a line chart showing total goals and average goals per match across all World Cups from 1930 to 2022.

Steps:

1. User navigates to the (`http://localhost:5001`).
2. Frontend JavaScript calls `GET /api/goals-per-worldcup`.
3. Flask routes the request to the data processor.
4. `data_processor.py` reads `matches.csv`, computes aggregates, and returns JSON.
5. `Chart.js` renders the line chart on the canvas element.

2.2 Use Case 2: Compare Two Teams Head-to-Head

Description: The user selects two national teams and views a side-by-side statistical comparison.

Steps:

1. User selects two teams from the dropdown (populated via `GET /api/available-teams`).
2. Frontend calls `GET /api/team-comparison?team1=Brazil&team2=Argentina`.
3. The data processor filters match records for both teams and computes wins, goals, and head-to-head results.
4. The frontend renders a bar chart and summary statistics card.

2.3 Use Case 3: Explore Top Teams by Metric

Description: The user selects a ranking metric (wins, goals, or titles) and views the top N teams as a bar chart.

Steps:

1. User selects a metric and limit from the UI controls.
2. Frontend calls `GET /api/top-teams?metric=goals&limit=10`.
3. Flask returns ranked team data as JSON.
4. `Chart.js` renders the horizontal bar chart with labels and values.

2.4 Use Case 4: View Goal Distribution by Continent

Description: The user views a pie chart showing the proportion of World Cup goals scored by teams from each continent.

Steps:

1. loads and frontend calls `GET /api/goals-by-continent`.
2. Data processor maps each team to its continent and aggregates goal counts.
3. `Chart.js` renders the pie chart.

3 Design Overview

The FIFA World Cup Analytics follows a classic three-tier web architecture: a data processing backend, a REST API middleware layer (Flask), and a browser-based frontend. The application is designed for local or server-hosted deployment and has no authentication layer in this version.

3.1 Design Goals and Constraints

- **Simplicity:** The system should be straightforward to set up locally with minimal dependencies.
- **Responsiveness:** The must render correctly on desktop, tablet, and mobile viewports.
- **Correctness:** All statistical computations must be derived directly from the source CSV data without hardcoded values.
- **Constraint – No Database:** Persistent storage uses flat CSV files rather than a relational database, reducing infrastructure requirements.
- **Constraint – No Authentication:** The current version does not implement user login or role-based access control.
- **Tool Constraint:** Backend must use Python 3.8+ and Flask 3.0. Frontend visualizations use Chart.js 4.0.

3.2 Design Assumptions

- The CSV data files (`matches.csv`, `tournaments.csv`) are considered the authoritative data source and are treated as read-only at runtime.
- The application is deployed in a single-process environment; no multi-worker concurrency is assumed.
- All clients accessing the are modern browsers with JavaScript enabled.
- Continental mappings for national teams are statically defined within the data processor.

3.3 Significant Design Packages

The system is decomposed into the following packages:

- **app.py** – Flask application entry point. Defines all REST API routes and wires them to data processor functions. Acts as the controller layer.
- **data_processor.py** – Analytics and data transformation layer. Loads CSV data using Pandas, performs all filtering, aggregation, and metric computation. Returns plain Python dictionaries serialised to JSON by Flask.
- **templates/index.html** – Single-page template. Rendered once on first load; all subsequent data is fetched via AJAX.

- **static/js/app.js** – Client-side logic. Manages Chart.js instances, API calls, and UI state.
- **static/css/styles.css** – Visual design layer. Implements a dark football-inspired theme with glassmorphism card styling.
- **data/** – Static CSV data store containing `matches.csv` and `tournaments.csv`.

3.4 Dependent External Interfaces

Application / Module	Interface Name	Description
Frontend (app.js)	Chart.js CDN	The frontend depends on the Chart.js library loaded via CDN to render all visualisations.
Frontend (app.js)	Google Fonts CDN	The Inter typeface is loaded from Google Fonts for consistent typographic rendering.

3.5 Implemented Application External Interfaces

Interface Name	Module	Description
GET /api/goals-per-worldcup	app.py	Returns total goals and average goals per match for each World Cup year.
GET /api/top-teams	app.py	Returns ranked team data by wins, goals, or titles. Accepts metric and limit query params.
GET /api/goals-by-stage	app.py	Returns goal counts segmented by match stage (Group vs Knockout).
GET /api/goals-by-continent	app.py	Returns aggregated goals per continent.
GET /api/team-comparison	app.py	Returns head-to-head statistics for two specified teams.
GET /api/available-teams	app.py	Returns the list of all distinct team names in the dataset.

4 Logical View

4.1 Design Model

The application follows an MVC-inspired pattern adapted for a Flask single-page application context.

Key Classes and Modules:

- **FlaskApp (app.py)** – Acts as the controller. Registers six route handlers, each delegating computation to `DataProcessor`. Returns JSON responses using Flask's `jsonify`.

- **DataProcessor (data_processor.py)** – Acts as the model/service layer. On initialisation, loads `matches.csv` and `tournaments.csv` into Pandas DataFrames. Exposes the following public methods:
 - `get_goals_per_worldcup()` – Groups matches by year, sums goals, and computes match averages.
 - `get_top_teams(metric, limit)` – Aggregates team-level statistics and returns the top N by the chosen metric.
 - `get_goals_by_stage()` – Segments matches into group stage and knockout stage, then sums goals per segment.
 - `get_goals_by_continent()` – Maps teams to continents and aggregates goals.
 - `get_team_comparison(team1, team2)` – Filters match records involving either team and computes comparative statistics.
 - `get_available_teams()` – Returns a sorted list of unique team names.
- **(static/js/app.js)** – Acts as the view controller in the browser. On `DOMContentLoaded`, fires all initial API calls in parallel, instantiates `Chart.js` objects, and binds UI control event listeners.

4.2 Use Case Realization

UC1 – Goals Trend (Sequence):

1. Browser loads `index.html` via `GET /`.
2. `app.js`: `fetchGoalsTrend()` calls `GET /api/goals-per-worldcup`.
3. `app.py`: `goals_per_worldcup()` invokes `DataProcessor.get_goals_per_worldcup()`.
4. `data_processor.py`: Reads `DataFrame`, applies `groupby('year')`, computes totals and averages, returns dict.
5. `app.py`: Serialises result to JSON and returns HTTP 200.
6. `app.js`: Receives JSON, creates a `Chart.js` Line instance on `#goalsTrendChart` canvas.

UC2 – Team Comparison (Sequence):

1. User selects two teams; `app.js` calls `GET /api/team-comparison?team1=&team2=`.
2. `app.py`: Routes to `team_comparison()` handler, reads query params, calls `DataProcessor.get_team_comparison(team1, team2)`.
3. `data_processor.py`: Filters `DataFrame` rows where either team appears as home or away, computes wins, losses, draws, and goals for each team.
4. `app.py`: Returns JSON.
5. `app.js`: Updates the comparison bar chart and summary text panel.

5 Data View

5.1 Domain Model

The persistent data is represented by two CSV files:

- **Match** – Represents a single World Cup match. Attributes include year, stage, home team, away team, home goals, away goals, host country, and attendance.
- **Tournament** – Represents a single World Cup edition. Attributes include year, host country, winner, total goals, and number of matches.

Both entities are related by the **year** attribute (one Tournament to many Matches).

5.2 Data Model (Persistent Data View)

5.2.1 Data Dictionary

Field	Source File	Type	Description
year	matches.csv	Integer	The year in which the World Cup was held.
stage	matches.csv	String	Match stage (e.g., Group, Quarter-Final, Final).
home_team	matches.csv	String	Name of the home team.
away_team	matches.csv	String	Name of the away team.
home_goals	matches.csv	Integer	Goals scored by the home team in regulation time.
away_goals	matches.csv	Integer	Goals scored by the away team in regulation time.
host_country	matches.csv	String	Country hosting the World Cup for that edition.
attendance	matches.csv	Integer	Match attendance figure.
year	tournaments.csv	Integer	The year of the tournament (primary key).
host	tournaments.csv	String	Host nation for that edition.
winner	tournaments.csv	String	The winning nation.
total_goals	tournaments.csv	Integer	Total goals scored across all matches.
matches_played	tournaments.csv	Integer	Total number of matches in the tournament.

6 Exception Handling

- **Missing Query Parameters:** If required query parameters (**team1**, **team2**, **metric**) are absent from a request, Flask returns an HTTP 400 Bad Request response with a descriptive JSON error message.
- **Unknown Team Name:** If a team name supplied to `/api/team-comparison` or `/api/top-teams` does not exist in the dataset, the data processor returns an empty result set rather than raising an exception. The frontend displays a “No data available” message to the user.

- **CSV Load Failure:** If either CSV file is missing or malformed at startup, `data_processor.py` raises a `FileNotFoundError` or `pandas.errors.ParserError`. These propagate to Flask's default error handler, returning HTTP 500. The error is logged to standard output.
- **Invalid Metric Value:** If the `metric` parameter for `/api/top-teams` is not one of `wins`, `goals`, or `titles`, the API returns HTTP 400 with a message listing valid options.
- **General Exceptions:** All route handlers are wrapped in try-except blocks. Any unexpected exception logs the traceback and returns a generic HTTP 500 JSON response, preventing raw stack traces from being exposed to the client.

7 Configurable Parameters

Parameter Name	Definition and Usage	Dy-namic?
HOST	The host address Flask binds to. Default: 0.0.0.0. Set in <code>app.py</code> .	No
PORT	The port Flask listens on. Default: 5001. Set in <code>app.py</code> .	No
DEBUG	Enables Flask debug mode and live reload. Default: <code>True</code> for development. Should be <code>False</code> in production.	No
MATCHES_CSV_PATH	Relative path to the matches data file. Default: <code>data/matches.csv</code> .	No
TOURNAMENTS_CSV_PATH	Relative path to the tournaments data file. Default: <code>data/tournaments.csv</code> .	No
DEFAULT_TEAM_LIMIT	Default number of teams returned by <code>/api/top-teams</code> when no <code>limit</code> param is supplied. Default: 10.	No

8 Quality of Service

8.1 Availability

The application is a locally hosted or single-server web application with no distributed components. There is no high-availability requirement for the current version. Availability depends on the host machine uptime. Since all data is loaded from CSV files at startup into in-memory DataFrames, there is no database dependency that could cause runtime unavailability. Restarting the Flask process is sufficient to recover from any crash.

8.2 Security and Authorization

The current version does not implement authentication or authorization. All API endpoints are publicly accessible to any client that can reach the server. The following baseline security considerations apply:

- Query parameters are validated for type and allowed values before being passed to the data processor, mitigating injection risks at the data layer.

- No user data is collected or stored.
- In a production deployment, the application should be placed behind a reverse proxy (e.g., Nginx) and Flask's debug mode must be disabled.

8.3 Load and Performance Implications

- The dataset covers 22 World Cups and approximately 250+ matches, which is small enough that all Pandas operations complete in well under 100ms per request on commodity hardware.
- No caching layer is implemented in this version. Each API call recomputes aggregations from the in-memory DataFrame. Given the dataset size, this is acceptable.
- The frontend loads all chart datasets in parallel on page load, minimising perceived latency.
- For a production deployment with a significantly larger dataset, computed aggregations should be cached (e.g., using Flask-Caching) to avoid redundant computation per request.

8.4 Monitoring and Control

- Flask's built-in development server logs all incoming HTTP requests, response codes, and latency to standard output.
- Application errors and exceptions are logged to standard output via Python's default exception traceback.
- No external monitoring integrations (e.g., Prometheus, Sentry) are implemented in the current version.
- In a production deployment, structured logging and an external monitoring service should be integrated to track error rates, endpoint latency, and uptime.